

PIR for Merkle Trees

Pranav

May 2026

1 Introduction

Bitcoin [?] is a recent *digital cryptocurrency* which uses a technology called *blockchain*. *Transactions* in the blockchain are stored by *blockchain nodes* using a *Merkle tree*. Given a transaction, a *client* can verify if the transaction is in the blockchain by retrieving a path in the Merkle tree from a blockchain node. However, the blockchain node learns the transaction the client is interested in, failing to offer client privacy.

More generally, in several applications [?], a *server* stores a binary tree, and a client wants to privately retrieve values in a path of the tree. A promising way to achieve this is to use a standard cryptographic primitive called *Private Information Retrieval* (PIR) [?]; Specifically, the values in the tree are arranged in a list, and the client retrieves values in the tree path one by one by using PIR.

However, this approach requires several rounds of client-server interaction and server computation proportional to the number of values in the path (this is because the server needs to perform computation proportional to the size of the database for each PIR invocation [?]). Instead, the client can batch the PIR queries using Batch Private Information Retrieval [?] to retrieve all the values in the Merkle path using a single round of client-server interaction. Even if we use the state-of-the-art Batch PIR scheme [?], this approach has costly communication. For instance, to privately retrieve a path in a Merkle tree consisting of 2^{20} leaves, each containing a value of size 32 bytes, the communication cost is **Pranav: Fill** MB. At a high level, this is because the generic batch PIR approach does not leverage the relationship between the positions of values the client wants to retrieve from the server.

A recent work [?] also targets the problem of privately retrieving tree paths, and leverages the fact that there is a nice relationship between the positions of values retrieved. They improve the server storage and server computation costs by small constant factors. However, communication remains costly.

1.1 Our Contributions

2 Preliminaries

2.1 Somewhat Homomorphic Encryption

Our scheme uses two *Somewhat Homomorphic Encryption* (SHE) schemes BFV [?] and RGSW [?]. We now present only the relevant details of SHE.

A Somewhat Homomorphic Encryption [?] scheme supports *encryption* and *decryption* operations. We describe how encryption is carried out in BFV and RGSW.

BFV encryption For some integer n which is a power of 2, let R be a polynomial ring $R = \mathbb{Z}[x]/(x^n + 1)$ and R_m denotes $R \bmod m$. The BFV encryption scheme is parameterized by two values t and q .

A plaintext message m is the polynomial R_t . Let a be a random polynomial in R_q , s be a polynomial with small coefficients in R , and e be a *noise* polynomial in R whose coefficients are sampled from a bounded Gaussian distribution. Then, the encryption of message m is the pair of polynomials $(a, -as + e + \Delta m)$, where Δ is a *scaling factor*.

RGSW encryption The RGSW scheme is parameterized by two values B and l . A *gadget vector* g^l is defined as follows:

$$g^l = (B^{\log q / \log B - 1}, B^{\log q / \log B - 2}, \dots, B^{\log q / \log B - l})$$

Encryption uses a *gadget matrix*, which is:

$$G = I_2 \vee g = \begin{bmatrix} g^l & 0 \\ 0 & g^l \end{bmatrix} \in R^{2l \times 2}$$

Then, an RGSW encryption of the polynomial $m \in R_t$ is:

$$C = Z + m \cdot G,$$

where each row of $Z \in R^{2l \times 2}$ is a BFV encryption of 0.

Operations The BFV and RGSW SHE schemes additionally support the following operations:

- **CtCtAdd**(c_1, c_2): This operation takes two BFV ciphertexts $c_1 \in R_q^2$, encrypting m_1 , and $c_2 \in R_q^2$ encrypting m_2 and outputs a BFV ciphertext encrypting $m_1 + m_2$.
- **CtPtMul**(c, p): This operations takes a BFV ciphertext $c \in R_q$ encrypting $m \in R_t$, and a plaintext $p \in R_t$ and outputs a BFV ciphertext encrypting $p \cdot m$.

- **CtCtMul**(c_1, c_2): This operation takes two BFV ciphertexts $c_1 \in R_q^2$, encrypting m_1 , and $c_2 \in R_q^2$, encrypting m_2 , and outputs a ciphertext encrypting $m_1 \cdot m_2$.
- **CtRotate**(c, r): This operation takes a BFV ciphertext c encrypting a plaintext vector $v = [v_1, v_2, \dots, v_m]$ and a value $r \in (0, m]$ and outputs $v^r = [v_{m-r+1}, v_{m-r+2}, \dots, v_m, v_1, v_2, \dots, v_{m-r}]$.
- **ExtPdt**(c, C): This operation takes a BFV ciphertext c encrypting a message m_1 , a RGSW ciphertext C encrypting a message m_2 and outputs a BFV ciphertext encrypting $m_1 \cdot m_2$.

BFV Decryption Each of the above operations increases the noise in the resultant ciphertext. A ciphertext can be decrypted using the secret s to obtain the underlying plaintext, provided the noise in the ciphertext is small.

3 Our Construction

3.1 Setup

Notation. Let there be a binary tree with N leaves. Each node in the tree contains a value that occupies g slots of the polynomial R_t . The height of the tree $L = \log N$. For $l \in [0, \log N]$, the l -th level of the tree contains 2^l nodes, indexed from 0 to $2^l - 1$. The value in the i -th node at the l -th level is denoted as $M[l][i]$.

The *path* of a leaf node i is the set of nodes $\mathcal{P}[i]$: i at level $\log N$, $\lfloor i/2 \rfloor$ at level $\log N - 1$, $\lfloor i/4 \rfloor$ at level $\log N - 2$, and so on up to node 0 at level 0. $\mathcal{P}[i][l]$ denotes the node at level l in the path.

Arranging the nodes in each level of the tree as a hypercube. Recall that n is the degree of the plaintext polynomial. Further, let $N_0 \in [0, N/n]$ be a flexible parameter.

For each $l \in [0, \log N]$, the l -th level of the tree can be arranged in a multi-dimensional hypercube. The nodes in each level l are arranged differently depending on whether the number of nodes in that level (2^l) is more than $n \cdot N_0$ or less than or equal to $n \cdot N_0$. We specify each case one after the other.

If $2^l > n \cdot N_0$, the nodes are arranged in a multidimensional hypercube with number of dimensions $2 + \log(2^l / (n \cdot N_0))$. The sizes of the dimensions are $n \times N_0 \times 2 \times \dots \times 2$. Let $rem = \log(2^l / (n \cdot N_0))$. Then, the node at position i is located at position $\gamma, \alpha, \beta_0, \beta_1, \dots, \beta_{rem-1}$, where $\gamma = \lfloor ni/N \rfloor$, $\alpha = \lfloor (i \bmod (N/n)) / (N/(n \cdot N_0)) \rfloor$, and $\beta_0, \beta_1, \dots, \beta_{rem-1}$ are the bits of $\beta = (i \bmod (N/n)) \bmod (N/(n \cdot N_0))$.

Otherwise, if $2^l < n \cdot N_0$, the values in level l are arranged as a square. The sizes of the dimensions are $n \times 2^l / n$. Each node i is located at a position γ, α , where $\gamma = \lfloor ni/2^l \rfloor$ and $\alpha = \lfloor i \bmod 2^l / n \rfloor$.

3.2 Server Preprocessing

The server packs the Merkle tree entries in plaintext polynomials in R_t . Let g denote the number of slots in the polynomial each entry occupies. Then, the server packs the nodes in the tree into polynomials as follows:

1. For each level $l \in [\log n, \log N]$:
 - The nodes are packed into $R = 2^l/n$ polynomials.
 - For each $p \in [R]$, pack the p -th set of n nodes, at positions $p, R + p, 2R + p, \dots, (n-1)R + p$, in the plaintext polynomials $\text{DB}^l[p][0], \text{DB}^l[p][1], \dots, \text{DB}^l[p][g-1]$. Specifically, the first $\log t$ bits of each of $M[l][p], M[l][R+p], M[l][2R+p], \dots, M[l][(n-1)R+p]$ are packed in one slot of $\text{DB}^l[p][0]$, the second $\log t$ bits of each of $M[l][p], M[l][R+p], M[l][2R+p], \dots, M[l][(n-1)R+p]$ are packed in one slot of $\text{DB}^l[p][1]$, and so on.

3.3 Client Query

The client has the position of a leaf node i and desires to retrieve the values of nodes in the path $\mathcal{P}[i]$. To retrieve this privately, the client computes the position of the leaf node in its hypercube (refer to Section 3.1), encrypts it homomorphically, and sends it to the server.

Specifically, the client computes a query as follows:

1. Node i is at position $\gamma = \lfloor ni/N \rfloor$, $\alpha = \lfloor (i \bmod (N/n)) / (N/(n \cdot N_0)) \rfloor$ (that lies from 0 to n), and $\beta = (i \bmod (N/n)) \bmod (N/(n \cdot N_0))$.
2. Let $\alpha_0, \alpha_1, \dots, \alpha_{N_0-1}$ be the bits in the one-hot representation of α , $\gamma_0, \gamma_1, \dots, \gamma_{n-1}$ be the bits in the one-hot representation of γ and $\beta_0, \beta_1, \dots, \beta_{rem-1}$ be the bits in β .
 - The client packs $\alpha_0, \alpha_1, \dots, \alpha_{N_0-1}$ and $\beta_0, \beta_1, \dots, \beta_{rem-1}$ in a single BFV ciphertext.
 - The client packs $\gamma_0, \gamma_1, \dots, \gamma_{n-1}$ in a BFV ciphertext in the evaluation form.
3. The client sends the query ciphertexts to the server.

3.4 Server Response

Using the client query, the server computes a response ciphertext for each level, combines them, and sends the result to the client. In detail, this is performed as follows:

1. For levels of the tree $l \in [0, \log N]$, for all $i \in [0, 2^l)$, return $M[l][i]$.
2. Initialize `res` to be the zero polynomial.
3. Expand the client query ciphertexts to obtain:

- $\text{BFV}_{\alpha_0}, \text{BFV}_{\alpha_1}, \dots, \text{BFV}_{\alpha_{N_0-1}}$.
 - $\text{GSW}_{\beta_0}, \text{GSW}_{\beta_1}, \dots, \text{GSW}_{\beta_{rem-1}}$.
4. For each level l from $\log N$ to $\log n + \log N_0$:
- Let $T = 2^l/N_0$. For each $i \in [0, T), k \in [0, g)$, $F[i \bmod N_0][k] = \text{CtCtAdd}(F[i \bmod N_0][k], \text{CtPtMul}(\text{BFV}_{\alpha_{i/N_0}}[i], DB^l[i][k]))$
 - For each $m \in [0, rem)$, for each $j \in [0, 2^{m-1})$, for each $k \in [0, g)$, $F[j][k] = \text{ExtPdt}(F[j][k], 1 - \text{GSW}_{\beta_m}) + \text{ExtPdt}(F[2^{m-1}+j][k], \text{GSW}_{\beta_m})$
 - For each $k \in [0, g)$, rotate $\text{res} = \text{CtRotate}(\text{res}, 1) + \text{CtCtMul}(F[0][k], \gamma)$.
5. For each level l from $\log n + \log N_0 - 1$ to $\log n$:
- Let $\text{diff} = 2^{(\log n + \log N_0 - 1) - l}$
 - For $j \in [0, N_0/(2\text{diff}))$ and $j \bmod 2\text{diff} = 0$, $\text{BFV}_{\alpha_j} = \text{CtCtAdd}(\text{BFV}_{\alpha_j}, \text{BFV}_{\alpha_j + \text{diff}})$
 - For each $k \in [0, g)$, $F[0][l] = \Sigma_{j \in [2^l]} \text{CtPtMul}(\text{BFV}_{\alpha_{j \cdot \text{diff}}}, DB^l[j][l])$.
 - For each $k \in [0, g)$, rotate $\text{res} = \text{CtRotate}(\text{res}, 1) + \text{CtCtMul}(F[0][k], \gamma)$.

Lemma 1. *Each node in the path of leaf node i from level $\log N$ to level $\log n$ is in segment number $\gamma = \lfloor ni/N \rfloor$.*

Proof. Consider any node on the path of leaf node i at level $\log N - a$, it is in segment $\lfloor (n \lfloor \dots \lfloor \lfloor i/2 \rfloor / 2 \rfloor \dots) \rfloor / (N/2^a) \rfloor$. Then, we need to show that $\lfloor (n \lfloor \dots \lfloor \lfloor i/2 \rfloor / 2 \rfloor \dots) \rfloor / (N/2^a) \rfloor$ is equal to γ .

To show this, we first show that:

$$\lfloor \lfloor i/2^k \rfloor / 2 \rfloor = \lfloor i/2^{k+1} \rfloor$$

Let $i/2^k = n + r$, where $0 \leq r < 1$. Then,

$$\lfloor \lfloor i/2^k \rfloor / 2 \rfloor = \lfloor \lfloor (n + r) \rfloor / 2 \rfloor = \lfloor n/2 \rfloor$$

Then, from the RHS, we have:

$$\lfloor i/2^{k+1} \rfloor = \lfloor (n + r)/2 \rfloor$$

Also, we have:

$$n/2 \leq (n + r)/2 < n/2 + 1/2$$

$$\lfloor n/2 \rfloor \leq \lfloor (n + r)/2 \rfloor < \lfloor n/2 + 1/2 \rfloor$$

If $f = \lfloor n/2 \rfloor$, then $\lfloor n/2 + 1/2 \rfloor < f + 1$, which means:

$$f \leq \lfloor (n + r)/2 \rfloor < f + 1$$

As $\lfloor (n + r)/2 \rfloor$ is an integer,

$$\lfloor n/2 \rfloor = f = \lfloor (n + r)/2 \rfloor$$

□

Lemma 2. Consider two values $a = i \bmod (N/n)$ and $b = \lfloor i/2 \rfloor \bmod (N/2n)$. Then, $b = \lfloor a/2 \rfloor$

Proof. We have that $i = q(N/n) + a$, where q is an integer in $[0, N/n - 1]$. Dividing both sides by 2:

$$i/2 = q(N/2n) + a/2$$

Performing the greatest integer operation on both sides:

$$\lfloor i/2 \rfloor = q(N/2n) + \lfloor a/2 \rfloor$$

The above equation is due to the fact that $q(N/2n)$ is an integer.

Then, by performing the modulo $N/2n$ operation on both sides, we get the lemma. $\square$

Lemma 3. Given any leaf node i , and level l , let α_l be the chunk number of $\mathcal{P}[i][l]$. For all $l \in (\log n + \log N_0, \log N]$, the chunk number α_l is equal to $\alpha_{\log N}$. For all $l \in (\log n, \log n + \log N_0)$, the chunk number $\alpha_{l-1} = \lfloor \alpha_l/2 \rfloor$.

Proof. Case: $l \in (\log n + \log N_0, \log N]$

Consider the values $\alpha_l = \lfloor i \bmod (2^l/n)/(2^l/n \cdot N_0) \rfloor$ and $\alpha_{l-1} = \lfloor \lfloor i/2 \rfloor \bmod 2^{l-1}/n / (2^{l-1}/n \cdot N_0) \rfloor$. Using Lemma 2, we have that:

$$\alpha_l = \lfloor (r \cdot n \cdot N_0)/2^l \rfloor$$

and,

$$\alpha_{l-1} = \lfloor \lfloor r/2 \rfloor (n \cdot N_0)/2^{l-1} \rfloor$$

, where r is an integer from $[0, 2^l/n)$.

If $r \bmod 2 = 0$, it is easy to see that $\alpha_l = \alpha_{l-1}$. Now, consider the case where $r \bmod 2 = 1$. Then,

$$\alpha_{l-1} = \lfloor (r-1)(n \cdot N_0)/2^{l-1} \rfloor$$

Let $r = 2a + 1$, $a = 2^{b-1}q + r$ and $b = (n \cdot N_0)/N$ then,

$$\begin{aligned} \alpha_l &= \lfloor (2a+1) \frac{1}{2^b} \rfloor \\ &= \lfloor q + (\frac{r+1/2}{2^{b-1}}) \rfloor \\ &= q \end{aligned}$$

Using $a = 2^{b-1}q + r$

$$\begin{aligned} \alpha_{l-1} &= \lfloor (2a) \frac{1}{2^b} \rfloor \\ &= \lfloor q + (\frac{r}{2^{b-1}}) \rfloor \\ &= q \end{aligned}$$

Case: $l \leq \log n + \log N_0$

We have that $\alpha_l = i \bmod (2^l/n)$ and $\alpha_{l-1} = \lfloor i/2 \rfloor \bmod (2^{l-1}/n)$. Using Lemma 2,

$$\alpha_{l-1} = \lfloor \alpha_l/2 \rfloor$$

□

Lemma 4. *Given any leaf node i , and level l , let β_l be the position of $\mathcal{P}[i][l]$ in its chunk. For all $l \in (\log n + \log N_0, \log N]$, $\beta_{l-1} = \lfloor \beta_l/2 \rfloor$.*

Proof. $\beta_l = (i \bmod (N/n)) \bmod (2^l/n^2)$ and $\beta_{l-1} = (\lfloor i/2 \rfloor \bmod (N/n)) \bmod (2^{l-1}/n^2)$. Applying Lemma 2, we get,

$$\beta_l = r \bmod (2^l/n^2), \beta_{l-1} = \lfloor (r/2) \rfloor \bmod (2^{l-1}/n^2).$$

Applying Lemma 2 again, we get the lemma statement. □